```
  def write_header_on(output)
    # ...
  end

  def write_body_on(output)
    # ...
  end

  def write_totals_on(output)
    # ...
  end
end
```

This defaulting mechanism is how Ruby handles private methods. Private methods may *not* be called with a receiver other than self, so they must be methods available in the current object. In the previous example, we'd might want to make the helper methods private, because they shouldn't be called from outside the InvoiceWriter class:

```
class InvoiceWriter
  def initialize(order)
    @order = order
  end

  def write_on(output)
    write_header_on(output)
    write_body_on(output)
    write_totals_on(output)
  end

  private def write_header_on(output)
    # ...
  end

  private def write_body_on(output)
    # ...
  end

  private def write_totals_on(output)
    # ...
  end
end
```

If the method name ends in =, and only if the method name ends in =, Ruby allows you to call the method as the left side of an assignment statement. Ruby allows you to place whitespace between the rest of the method name and the closing =. The last two lines of this example are equivalent:

```
class Person
  def name=(new_name)
    @name = new_name
  end
end

p = Person.new
p.name = "Brandi Carlile"
p.name=("Elton John")
```